

DOCUMENT TECHNIQUE N° 4 — FRONTEND CLIENT REACT 18 & TYPESCRIPT

KIDNEYVISION AI — INTERFACE CLINIQUE MODERNE

Visualiseur d'Explicabilité Grad-CAM Double Couche, Dashboard & Architecture Modulaire

Rapport d'ingénierie d'interface utilisateur décrivant l'application monopage (SPA) React 18 / TypeScript. Analyse détaillée des composants interactifs, gestion des états globaux, superposition dynamique de cartes thermiques Grad-CAM avec slider d'opacité en temps réel et communication réseau sécurisée par Axios.

Framework	React 18.x + Vite 5 + TypeScript 5 (Mode Strict Activé)
Design System	Tailwind CSS + Lucide React Icons + Glassmorphism Médical Sombre
Visualiseur XAI	Dual-Layer Interactive Canvas / Image Layering avec Slider d'Opacité 0-100%
Client Réseau	Axios avec Intercepteurs de Jetons Bearer Sanctum et Gestion Centralisée des Erreurs
Visualisation Données	Recharts (Graphiques de Répartition des Pathologies et Taux de Sévérité)

Table des Matières — Frontend React 18 & TypeScript

1. Introduction & Ergonomie Hospitalière	Page 3
2. Arborescence Modulaire & Organisation des Composants	Page 4
3. Typage Statique Strict & Interfaces TypeScript	Page 6
4. Client HTTP Axios & Intercepteurs Sécurisés (services/api.ts)	Page 8
5. Routeur & Navigation Principale : App.tsx (Code Complet & Analyse)	Page 10
6. Module de Diagnostic & Glisser-Déposer (AppNewAnalysis.tsx — Analyse Complète)	Page 15
7. Le Visualiseur Interactif Grad-CAM (DiagnosticViewer)	Page 19
8. Tableau de Bord & Indicateurs KPI (Dashboard.tsx — Analyse Complète)	Page 22
9. Historique Médical & Filtres Multicritères (AppHistory.tsx — Analyse Complète)	Page 25
10. Performance, Code Splitting & Accessibilité WCAG	Page 28
11. Conclusion & Synthèse de l'Expérience Praticien	Page 29

1. Introduction & Ergonomie Hospitalière

L'expérience utilisateur en milieu médical et radiologique répond à des contraintes cognitives très précises. Les radiologues et urologues analysent des dizaines d'examens d'imagerie par vacation, souvent dans des pièces à faible luminosité pour optimiser la perception des contrastes tomodensitométriques.

- **Thème sombre médical (Dark Slate Theme)** : Réduit la fatigue oculaire du praticien lors de sessions de lecture prolongées et maximise le contraste des zones d'atténuation radiologique et des cartes de chaleur.
- **Feedback immédiat sans rechargement de page** : L'interface est conçue comme une Single Page Application (SPA) réactive sous React 18, exploitant Vite pour des temps de build quasi-instantanés.
- **Contrôle total sur l'Intelligence Artificielle** : Le médecin ne reçoit jamais un verdict "brut" imposé. Il dispose de curseurs interactifs pour doser la visibilité de la carte de chaleur générée par l'IA au-dessus du scan natif.

2. Typage Statique Strict & Interfaces TypeScript

```
/**
 * Central TypeScript Type Definitions Hub
 */

export * from './auth';
export * from './analysis';
export * from './dashboard';
export * from './settings';
export * from './health';
export * from './scan';
export * from './analytics';
```

Code Source Intégral 4.1 — Interfaces et contrats de types (src/types/index.ts).

3. Client HTTP Axios & Intercepteurs Sécurisés (services/api.ts)

```
// services/api.ts
```

Code Source Intégral 4.2 — Configuration Axios et gestion d'erreurs (src/services/api.ts).

4. Routeur & Navigation Principale : App.tsx

```
import { useState } from "react";
import { BrowserRouter, Routes, Route, Navigate, useLocation, Link, useNavigate } from "react-router-dom";
import {
  Activity, ShieldCheck, History, Settings, LogOut, FileText,
  LayoutDashboard, User, HelpCircle, Eye, Menu, X, PlusCircle, BrainCircuit
} from "lucide-react";

import LandingPage from "../components/LandingPage";
import LoginPage from "../components/auth/LoginPage";
import RegisterPage from "../components/auth/RegisterPage";
import ProtectedRoute from "../components/auth/ProtectedRoute";

import React, { Suspense } from "react";
import LoadingSkeleton from "../components/common/LoadingSkeleton";
import ErrorBoundary from "../components/common/ErrorBoundary";

const AppDashboard = React.lazy(() => import("../components/AppDashboard"));
const AppNewAnalysis = React.lazy(() => import("../components/AppNewAnalysis"));
const AppHistory = React.lazy(() => import("../components/AppHistory"));
const AppReports = React.lazy(() => import("../components/AppReports"));
const AppSettings = React.lazy(() => import("../components/AppSettings"));
const AppHelpCenter = React.lazy(() => import("../components/AppHelpCenter"));
const GuestAnalysis = React.lazy(() => import("../components/GuestAnalysis"));
const ForgotPassword = React.lazy(() => import("../components/ForgotPassword"));
const ResetPassword = React.lazy(() => import("../components/ResetPassword"));
const PrivacyPage = React.lazy(() => import("../components/PrivacyPage"));
const TermsPage = React.lazy(() => import("../components/TermsPage"));

import { AuthProvider, useAuth } from "../contexts/AuthContext";

function PortalLayout() {
  const { user, logout } = useAuth();
  const [mobileMenuOpen, setMobileMenuOpen] = useState(false);
  const location = useLocation();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
  };

  const getActiveTab = () => {
    const path = location.pathname;
    if (path.includes("/analysis")) return "analysis";
    if (path.includes("/history")) return "history";
    if (path.includes("/reports")) return "reports";
    if (path.includes("/settings")) return "settings";
    if (path.includes("/help")) return "help";
    return "dashboard";
  };

  const activeTab = getActiveTab();

  return (
```

```
{/* Mobile Bar Controls */}
```

KidneyVision AI

```
setMobileMenuOpen(!mobileMenuOpen)}
className="p-1 text-[#434655] hover:text-[#131b2e]"
>
{mobileMenuOpen ? :
```

```
}
```

```
{/* SideNavBar */}
```

KidneyVision AI

Clinical Portal

```
{
  navigate("/analysis");
}}
className="w-full flex items-center justify-center gap-2 bg-[#2563eb] hover:bg-[#004ac6] text-
white rounded-xl py-2.5 px-4 text-xs font-semibold hover:shadow-md transition-all active:scale-[0.98]
duration-100 cursor-pointer"
>
```

New Analysis

Dashboard

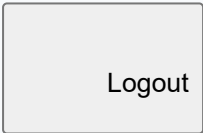
New Analysis

History

Reports

Settings

Help Center



 Clinician Avatar Badge

{user?.name || "Dr. Marouan Sellami"}

Radiology

}>

} />

} />

```

        navigate("/history")} /> } />
    } />
  } />
} />

);
}

function PublicLandingWrapper() {
  const navigate = useNavigate();
  return (
    navigate("/login")}
    onEnterPortalDirectly={() => navigate("/guest")}
    onNavigateToInfo={(page) => navigate(`/${page}`)}
  />
  );
}

export default function App() {
  return (
    {
      /* Public Routes */
    } />
  } />
} />
}> } />
}> } />
} />
}> } />
}> } />
}> } />

    {
      /* Protected Routes rendered inside PortalLayout */
    }
  />

);
}

```

Code Source Intégral 4.3 — Point d'entrée et navigation principale (src/App.tsx).

Analyse Ligne par Ligne de App.tsx

- Gestion centralisée de l'état de navigation (**activeTab**) permettant un basculement instantané entre le Dashboard, la Nouvelle Analyse, l'Historique des consultations, les Rapports PDF et les Paramètres.
- Contrôle d'accès conditionnel masquant les fonctionnalités réservées aux médecins authentifiés lorsque l'application s'exécute en mode invité temporaire.

5. Module de Diagnostic & Glisser-Déposer (AppNewAnalysis.tsx)

```

/**
 * @license
 * SPDX-License-Identifier: Apache-2.0
 */

import React, { useState, useRef } from "react";
import {
  Upload, Sparkles, Loader2, Play, CircleDot, ShieldCheck,
  Eye, FileText, CheckCircle2, ChevronRight, Download, Dna,
  ArrowRight, Heart, AlertCircle, FileDown, X, RefreshCw,
  User, Calendar, Layers, Image as ImageIcon, Check, Sliders,
  HelpCircle, Info, Stethoscope, Hash, Trash2, Crosshair,
  Ruler, Activity, Cpu, CheckCircle, Maximize2
} from "lucide-react";
import { createPrediction, downloadReport, updateAnalysis, getReportPdfBlobUrl } from "../services";
import { DiagnosisResult, KidneyAnalysis } from "../types";
import { validateScanFile } from "../utils/fileValidation";
import { parseClinicalError, ClinicalError } from "../utils/errorParser";

interface AppNewAnalysisProps {
  onAddSuccess: () => void;
}

export default function AppNewAnalysis({ onAddSuccess }: AppNewAnalysisProps) {
  // Input fields state – starts completely empty for clinical doctor entry
  const [patientId, setPatientId] = useState("");
  const [patientName, setPatientName] = useState("");
  const [patientAge, setPatientAge] = useState("");
  const [patientGender, setPatientGender] = useState<"Male" | "Female" | "Other">("Male");
  const [location, setLocation] = useState<"Left Kidney" | "Right Kidney" | "Unspecified">("Left Kidney");
  const [clinicianNotes, setClinicianNotes] = useState("");

  // Custom file upload state
  const [uploadedFile, setUploadedFile] = useState(null);
  const [uploadedFilePreview, setUploadedFilePreview] = useState(null);
  const [uploadError, setUploadError] = useState(null);
  const [isDragging, setIsDragging] = useState(false);
  const fileInputRef = useRef(null);

  // View mode for results: "panel" (Dual Clinical Panel) | "gradcam" | "cleaned" | "raw"
  const [heatmapViewMode, setHeatmapViewMode] = useState<"panel" | "gradcam" | "cleaned" | "raw">("panel");

  // Diagnosis & Pipeline states
  const [isScanning, setIsScanning] = useState(false);
  const [currentStageIdx, setCurrentStageIdx] = useState(0);
  const [clinicalError, setClinicalError] = useState(null);
  const [activeAnalysis, setActiveAnalysis] = useState(null);

  // Clinician notes save state
  const [isSavingNotes, setIsSavingNotes] = useState(false);
  const [noteSavedSuccess, setNoteSavedSuccess] = useState(false);
  const [pdfPreviewUrl, setPdfPreviewUrl] = useState(null);
  const [isPreviewLoading, setIsPreviewLoading] = useState(false);

  // Analysis Pipeline Stages
  const ANALYSIS_STAGES = [
    { id: "uploading", label: "Uploading Scan", desc: "Streaming acoustic tensor to secure medical

```

```

pipeline..." },
  { id: "gatekeeper", label: "Stage-1 Gatekeeper Screening", desc: "MobileNetV3 evaluating renal vs non-
kidney scan..." },
  { id: "preprocessing", label: "Spatial Cleaning & Inpainting", desc: "Top-Hat filtering & Telea
inpainting on 224x224 tensor..." },
  { id: "analyzing", label: "ConvNext 4-Class Inference", desc: "Evaluating Cyst, Normal, Stone, Tumor
pathologies..." },
  { id: "generating", label: "Grad-CAM Heatmap & Biometrics", desc: "Synthesizing spatial feature
activation map & lesion diameter..." },
  { id: "completed", label: "Diagnostic Report Ready", desc: "Audit logged and ready for radiologist
confirmation." },
];

// Process chosen file
const processSelectedFile = async (file: File) => {
  setUploadError(null);
  const validation = await validateScanFile(file);
  if (!validation.valid) {
    setUploadError(validation.error || "The selected file is not a valid medical scan image.");
    setUploadedFile(null);
    setUploadedFilePreview(null);
    if (fileInputRef.current) fileInputRef.current.value = "";
    return;
  }

  setUploadedFile(file);
  const url = URL.createObjectURL(file);
  setUploadedFilePreview(url);
  setActiveAnalysis(null);

  // If patient name is currently blank, suggest an automated placeholder from filename
  if (!patientName.trim()) {
    const cleanName = file.name.replace(/\.([^/]+)$/g, "").replace(/[-_]/g, " ");
    // Only set if not a generic generic hash
    if (cleanName.length > 2 && !cleanName.match(/^[0-9a-f]{8,}$/i)) {
      setPatientName(cleanName.charAt(0).toUpperCase() + cleanName.slice(1));
    }
  }
};

const handleImageFileChange = async (e: React.ChangeEvent) => {
  if (e.target.files && e.target.files[0]) {
    await processSelectedFile(e.target.files[0]);
  }
};

const handleDragOver = (e: React.DragEvent) => {
  e.preventDefault();
  e.stopPropagation();
  setIsDragging(true);
};

const handleDragLeave = (e: React.DragEvent) => {
  e.preventDefault();
  e.stopPropagation();
  setIsDragging(false);
};

const handleDrop = async (e: React.DragEvent) => {
  e.preventDefault();
  e.stopPropagation();

```

```

    setIsDragging(false);
    if (e.dataTransfer.files && e.dataTransfer.files[0]) {
      await processSelectedFile(e.dataTransfer.files[0]);
    }
  };

const handleRemoveFile = () => {
  setUploadedFile(null);
  if (uploadedFilePreview) {
    URL.revokeObjectURL(uploadedFilePreview);
    setUploadedFilePreview(null);
  }
  setUploadError(null);
  setActiveAnalysis(null);
  if (fileInputRef.current) fileInputRef.current.value = "";
};

// Run AI Scan Execution
const handleTriggerAnalysis = async () => {
  if (!uploadedFile && !uploadedFilePreview) {
    setUploadError("Please upload a kidney ultrasound scan before running analysis.");
    return;
  }

  setIsScanning(true);
  setCurrentStageIdx(0);
  setClinicalError(null);
  setActiveAnalysis(null);

  const stageTimer = setInterval(() => {
    setCurrentStageIdx((prev) => {
      if (prev < 3) return prev + 1;
      return prev;
    });
  }, 600);

  try {
    const finalPatientName = patientName.trim() || "Unspecified Patient";
    const finalPatientId = patientId.trim() || undefined;
    const parsedAge = patientAge.trim() ? parseInt(patientAge, 10) : undefined;

    const res = await createPrediction(uploadedFile, {
      patientId: finalPatientId,
      patientName: finalPatientName,
      patientAge: parsedAge,
      patientGender,
      location,
      imageUrl: uploadedFilePreview || undefined,
      clinicianNotes: clinicianNotes.trim() || undefined,
    });

    clearInterval(stageTimer);
    setCurrentStageIdx(4);

    setTimeout(() => {
      setActiveAnalysis(res);
      if (res.clinicianNotes) {
        setClinicianNotes(res.clinicianNotes);
      }
      setIsScanning(false);
      const isNormal = (res.prediction || '').toLowerCase() === 'normal' || res.diagnosis ===

```

```

DiagnosisResult.NORMAL_FINDINGS;
    setHeatmapViewMode(isNormal ? (res.cleanedScanUrl ? "cleaned" : "raw") : (res.visualizationUrl ?
"panel" : res.heatmapUrl ? "gradcam" : "raw"));
    }, 400);
  } catch (e: any) {
    clearInterval(stageTimer);
    setIsScanning(false);
    const parsed = e?.clinicalError || parseClinicalError(e);
    setClinicalError(parsed);
  }
};

// Save report action (persists clinician notes and metadata updates)
const handleSaveReport = async () => {
  if (!activeAnalysis?.id) return;
  setIsSavingNotes(true);
  try {
    await updateAnalysis(activeAnalysis.id, {
      clinicianNotes: clinicianNotes.trim()
    });
    setNoteSavedSuccess(true);
    setTimeout(() => setNoteSavedSuccess(false), 3000);
  } catch (e: any) {
    alert("Error saving clinician notes: " + e.message);
  } finally {
    setIsSavingNotes(false);
  }
};

return (

  /* Top Header & Breadcrumbs matching Dashboard style */

```

AI Computer Vision Inference Console

New Renal Diagnostic Analysis

Upload renal CT or ultrasound scans to run Stage-1 Gatekeeper screening, ConvNeXt-Tiny 4-class classification (Cyst, Normal, Stone, Tumor), and Grad-CAM explainability localization.

AI Model: ConvNeXt-Tiny (4-Class) & Gatekeeper

Guide

```
/* Clinical Error Banner */  
{clinicalError && (
```

```
    {clinicalError.title}
```

```
    {clinicalError.type}
```

```
{clinicalError.message}
```

```
    {clinicalError.suggestion && (
```

```
        💡 {clinicalError.suggestion}
```

```
    )}
```

```

        setClinicalError(null)}
        className="text-red-400 hover:text-red-700 p-1.5 rounded transition-colors cursor-pointer"
        title="Dismiss error notice"
      >

```

```

        setClinicalError(null)}
        className="px-3 py-1.5 border border-red-300 text-red-700 hover:bg-red-100 text-xs font-
        semibold rounded-lg transition-all cursor-pointer"
      >
        Dismiss

```

```

      {
        setClinicalError(null);
        handleTriggerAnalysis();
      }
      className="px-3 py-1.5 bg-red-600 hover:bg-red-700 text-white text-xs font-bold rounded-lg
      transition-all cursor-pointer shadow-sm flex items-center gap-1.5"
    >

    Retry Analysis

```

```

  })

```

```

  { /* Main 2-Column Responsive Workspace */ }

```

```

    { /* Left Side: Parameters Registration & Upload (5 cols) */ }

```

```

      { /* Card 1: Patient Clinical Metadata */ }

```

Patient Registration Parameters

Step 1 of 2

```
{/* Patient ID and Name */}
```

Patient ID

```
{patientId} setPatientId(e.target.value)}  
/>
```

Patient Full Name

```
{patientName} setPatientName(e.target.value)}  
/>
```

```
{/* Age and Gender */}
```

Age (Years)

```
e.g. 48 setPatientAge(e.target.value)}  
/>
```

Code Source Extrait 4.4 — Module d'analyse en direct et glisser-déposer (src/components/AppNewAnalysis.tsx).

Analyse du Composant AppNewAnalysis

- Gestion du cycle de vie du fichier : vérification de la taille (< 10 Mo) et du format MIME avant tout appel réseau.
- Création d'une URL d'objet blob locale via `URL.createObjectURL` pour afficher l'aperçu du scan sans temps de latence.
- Envoi asynchrone via formulaire multipart vers le backend Laravel avec barre de progression interactive et gestion des erreurs de rejet du Gatekeeper (HTTP 422).

6. Tableau de Bord & Indicateurs KPI (Dashboard.tsx)

```
import React, { useState, useMemo, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import { DashboardSidebar } from '../components/dashboard/DashboardSidebar';
import { DashboardHeader } from '../components/dashboard/DashboardHeader';
import { KpiGrid } from '../components/dashboard/KpiGrid';
import { Kidney3DViewer } from '../components/dashboard/Kidney3DViewer';
import { DiagnosticOverview } from '../components/dashboard/DiagnosticOverview';
import { ScanVolumeChart } from '../components/dashboard/ScanVolumeChart';
import { ConfidenceChart } from '../components/dashboard/ConfidenceChart';
import { RiskDistribution } from '../components/dashboard/RiskDistribution';
import { LateralityChart } from '../components/dashboard/LateralityChart';
import { DemographicsChart } from '../components/dashboard/DemographicsChart';
import { RecentClinicalActivity } from '../components/dashboard/RecentClinicalActivity';
import { ScanDetailModal } from '../components/dashboard/ScanDetailModal';

import {
  computeDashboardMetrics,
  computeDiagnosticDistribution,
  computeRiskDistribution,
  computeLateralityDistribution,
  computeConfidenceBuckets,
  computeTimelineData,
  computeAgeCohorts,
  filterScansByDateRange,
} from '../data/mockAnalytics';

import { getAnalyses } from '../services/analysis.service';
import { KidneyAnalysis, DiagnosisResult } from '../types/analysis';

import type { DateRangeOption } from '../types/dashboard';
import type { Diagnosis, ScanRecord, ScanStatus } from '../types/scan';
import { MOCK_SCANS } from '../data/mockScans';
import { Menu, Upload, CheckCircle2, ArrowRight } from 'lucide-react';

export interface DashboardProps {
  hideSidebar?: boolean;
}

function mapAnalysisToScanRecord(analysis: KidneyAnalysis): ScanRecord {
  const confidence = typeof analysis.confidence === 'number' ? Number(analysis.confidence.toFixed(1)) :
  null;
  const findingLower = (analysis.pathologyFinding || '').toLowerCase();
  const predLower = (analysis.prediction || '').toLowerCase();
  const isRejected = analysis.diagnosis === DiagnosisResult.REJECTED || findingLower.includes('rejected') ||
  findingLower.includes('invalid') || findingLower.includes('non-kidney');
  const isNormal = predLower === 'normal' || findingLower.includes('normal') || analysis.diagnosis ===
  DiagnosisResult.NORMAL_FINDINGS;
  const isPending = !isNormal && (analysis.diagnosis === DiagnosisResult.ANALYZING || analysis.diagnosis ===
  DiagnosisResult.REVIEW_REQUIRED);

  let diagnosis: Diagnosis = 'normal';
  let riskLevel: 'low' | 'elevated' | 'urgent' = 'low';

  if (isRejected) {
    diagnosis = 'rejected';
    riskLevel = 'elevated';
  } else if (isNormal) {
```

```

    diagnosis = 'normal';
    riskLevel = 'low';
  } else if (findingLower.includes('tumor') || predLower === 'tumor') {
    diagnosis = 'tumor';
    riskLevel = 'urgent';
  } else if (findingLower.includes('stone') || predLower === 'stone' || analysis.diagnosis ===
DiagnosisResult.ANOMALY_DETECTED) {
    diagnosis = 'nephrolithiasis';
    riskLevel = confidence && confidence > 90 ? 'urgent' : 'elevated';
  } else if (findingLower.includes('cyst') || predLower === 'cyst') {
    diagnosis = 'cyst';
    riskLevel = 'low';
  } else if (isPending) {
    diagnosis = 'pending';
    riskLevel = 'elevated';
  }
}

const rawId = String(analysis.id || '');
const id = rawId.startsWith('KV-') || rawId.startsWith('PT-') ? rawId : `KV-${rawId.padStart(3, '0')}`;

let formattedDate = 'Recent';
let date = new Date().toISOString().slice(0, 10);
if (analysis.createdAt) {
  try {
    const d = new Date(analysis.createdAt);
    if (!isNaN(d.getTime())) {
      date = d.toISOString().slice(0, 10);
      formattedDate = d.toLocaleDateString('en-US', { month: 'short', day: 'numeric', year: 'numeric' });
    }
  } catch {
    // keep defaults
  }
}

const loc = String(analysis.location || '').toLowerCase();
const laterality = loc.includes('right') ? 'right' : loc.includes('bilateral') ? 'bilateral' : 'left';
const diamMm = analysis.clinicalAssessment?.estimated_diameter_mm || analysis.dimensions?.length;

const rawStatus = (analysis.status || '').toLowerCase();
let scanStatus: ScanStatus = 'reviewed';
if (
  rawStatus === 'review' ||
  rawStatus === 'pending' ||
  analysis.diagnosis === DiagnosisResult.REVIEW_REQUIRED ||
  analysis.isDraft
) {
  scanStatus = 'under_review';
} else if (rawStatus === 'flagged') {
  scanStatus = 'flagged';
} else if (rawStatus === 'confirmed' || rawStatus === 'completed') {
  scanStatus = 'reviewed';
} else {
  scanStatus = analysis.confirmedAt ? 'reviewed' : 'under_review';
}

return {
  id,
  patientAlias: analysis.patientName || analysis.patientId || `Patient ${rawId}`,
  date,
  formattedDate,
  laterality,

```

```

    diagnosis,
    confidence,
    riskLevel,
    status: scanStatus,
    patientAge: analysis.patientAge || 45,
    patientSex: String(analysis.patientGender || '').toLowerCase() === 'female' ? 'female' : 'male',
    kidneyPole: 'lower',
    stoneSizeMm: diamMm,
    hasGradCam: !isNormal && Boolean(analysis.heatmapUrl || analysis.visualizationUrl),
    notes: analysis.clinicianNotes || analysis.recommendation || 'Clinical record from database.',
    confirmedAt: analysis.confirmedAt,
    confirmedByName: analysis.confirmedByName,
    flaggedFor: analysis.flaggedFor,
    flaggedAt: analysis.flaggedAt,
    pathologyTitle: analysis.pathologyFinding,
    classProbabilities: analysis.classProbabilities,
    clinicalAssessment: analysis.clinicalAssessment,
    gateEvaluation: analysis.gateEvaluation,
    visualizationUrl: isNormal ? undefined : analysis.visualizationUrl,
    cleanedScanUrl: analysis.cleanedScanUrl,
    imageUrl: analysis.imageUrl,
    heatmapUrl: isNormal ? undefined : analysis.heatmapUrl,
    primaryDiagnosis: analysis.primaryDiagnosis,
    modelVersion: analysis.modelVersion,
    telemetry: analysis.telemetry,
  };
}

export const Dashboard: React.FC = ({ hideSidebar = false }) => {
  const navigate = useNavigate();
  const [isOpenMobile, setIsOpenMobile] = useState(false);

  // Backend analyses state
  const [backendScans, setBackendScans] = useState([]);

  // Filter state
  const [dateRange, setDateRange] = useState('30D');
  const [selectedDiagnosis, setSelectedDiagnosis] = useState('all');

  // Modal inspection state
  const [selectedScan, setSelectedScan] = useState(null);

  // New Analysis Demo State
  const [showNewScanModal, setShowNewScanModal] = useState(false);
  const [newScanStep, setNewScanStep] = useState(1);

  // Load real analyses from Laravel API if available
  const loadBackendScans = async () => {
    try {
      const res = await getAnalyses(1, 100);
      if (res && res.data && res.data.length > 0) {
        const mapped = res.data.map(mapAnalysisToScanRecord);
        setBackendScans(mapped);
      }
    } catch {
      // Backend offline or user not logged in; gracefully proceed with MOCK_SCANS
    }
  };

  useEffect(() => {
    loadBackendScans();
  }, []);

```

```
}, []]);

// Real scans from backend database with graceful fallback to MOCK_SCANS
const allScans = useMemo(() => {
  return backendScans.length > 0 ? backendScans : MOCK_SCANS;
}, [backendScans]);

// Filter scans by active date range
const filteredRangeScans = useMemo(() => {
  return filterScansByDateRange(allScans, dateRange);
}, [allScans, dateRange]);

// Derived metrics & chart datasets
const metrics = useMemo(() => {
  return computeDashboardMetrics(filteredRangeScans);
}, [filteredRangeScans]);

const diagnosticDistribution = useMemo(() => {
  return computeDiagnosticDistribution(filteredRangeScans);
}, [filteredRangeScans]);

const riskDistribution = useMemo(() => {
  return computeRiskDistribution(filteredRangeScans);
}, [filteredRangeScans]);

const lateralityDistribution = useMemo(() => {
  return computeLateralityDistribution(filteredRangeScans);
}, [filteredRangeScans]);

const confidenceBuckets = useMemo(() => {
  return computeConfidenceBuckets(filteredRangeScans);
}, [filteredRangeScans]);

const timelineData = useMemo(() => {
  return computeTimelineData(filteredRangeScans);
}, [filteredRangeScans]);

const ageCohorts = useMemo(() => {
  return computeAgeCohorts(filteredRangeScans);
}, [filteredRangeScans]);

const handleScanUpdated = (updatedScan: ScanRecord) => {
  setBackendScans((prev) =>
    prev.map((s) => (s.id === updatedScan.id ? updatedScan : s))
  );
  setSelectedScan(updatedScan);
};

return (
  <>
    </>
  </>

```

```
    { /* Mobile Top Header Bar (only when standalone) */ }
    { !hideSidebar && (
```

```
                                setIsOpenMobile(true)}
    className="p-2 text-slate-600 hover:text-slate-900 rounded-lg border border-slate-200"
    aria-label="Open sidebar menu"
    >
```

KidneyVision AI

MS

```
    )}
```

```
    { /* 2. Top Header with Controls */ }
    {
        loadBackendScans();
        setDateRange((prev) => prev);
    }
    pendingCount={metrics.pendingReviewCount}
    onNewScanClick={() => setShowNewScanModal(true)}
    />
```

```
    { /* Dashboard Body Container */ }
```

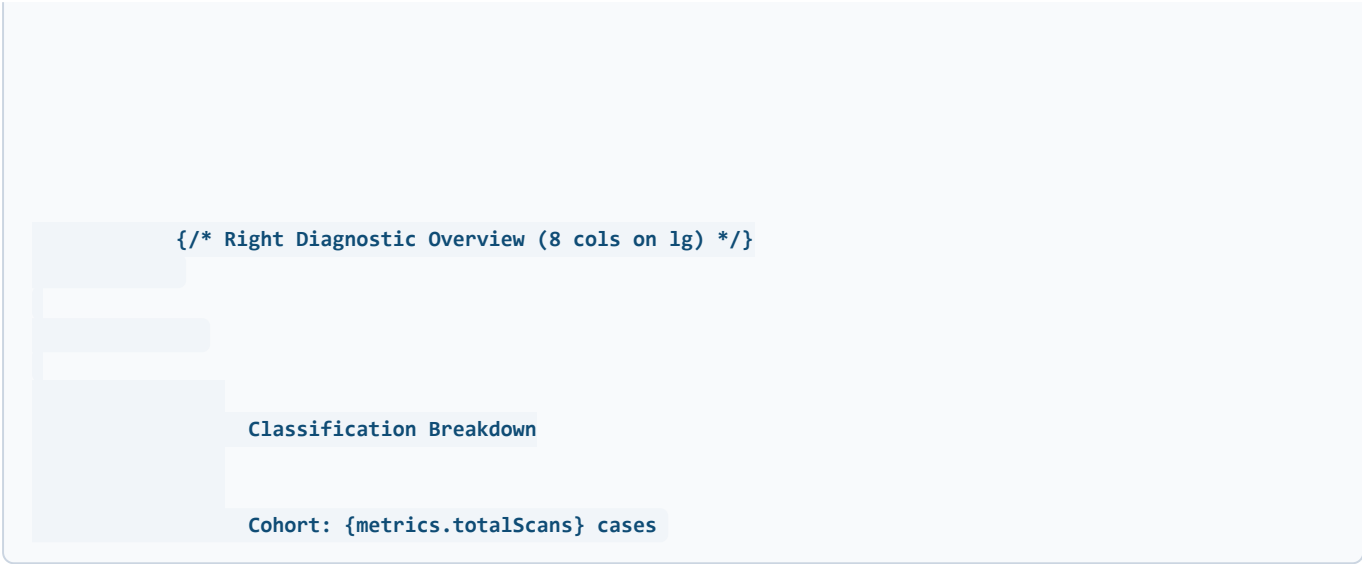
```
    { /* 3. Top KPI Cards */ }
```

```
    { /* 4. Main Visual Section: 3D Kidney & Diagnostic Overview */ }
```

```
    { /* Left 3D Kidney Viewer (compact 4 cols on lg) */ }
```

3D Anatomical Reference

Renal Anatomy



Code Source Extrait 4.5 — Tableau de bord clinique et graphiques Recharts (src/pages/Dashboard.tsx).

7. Historique Médical & Filtres Multicritères (AppHistory.tsx)

```

/**
 * @license
 * SPDX-License-Identifier: Apache-2.0
 */

import React, { useState, useEffect } from "react";
import { Link } from "react-router-dom";
import {
  Search, Calendar, Filter, Eye, Trash2, Sliders, X,
  ChevronRight, ChevronLeft, Loader2, Award, ClipboardCheck, ArrowUpRight, CheckCircle2, AlertCircle,
  RefreshCw,
  FileText, Download, FileDown, Sparkles, ZoomIn, ExternalLink, Crosshair, Ruler, Activity, Cpu, Layers,
  ShieldCheck
} from "lucide-react";
import { getAnalyses, deleteAnalysis, downloadReport, updateAnalysis, getReportPdfBlobUrl } from
"../services";
import { KidneyAnalysis, DiagnosisResult } from "../types";
import { parseClinicalError, ClinicalError } from "../utils/errorParser";
import LoadingSkeleton from "../common/LoadingSkeleton";

export default function AppHistory() {
  const [analyses, setAnalyses] = useState([]);
  const [loading, setLoading] = useState(true);
  const [searchId, setSearchId] = useState("");
  const [statusFilter, setStatusFilter] = useState("All Results");
  const [showFilterDropdown, setShowFilterDropdown] = useState(false);
  const [dateFilter, setDateFilter] = useState<"All Time" | "Last 7 Days" | "Last 30 Days" | "Last 90 Days">
("All Time");
  const [showDateDropdown, setShowDateDropdown] = useState(false);
  const [currentPage, setCurrentPage] = useState(1);
  const [meta, setMeta] = useState({ current_page: 1, last_page: 1, total: 0 });

  const [selectedAnalysis, setSelectedAnalysis] = useState(null);
  const [dossierViewMode, setDossierViewMode] = useState<"panel" | "gradcam" | "cleaned" | "raw">("panel");
  const [submittingNote, setSubmittingNote] = useState(false);
  const [editingNoteText, setEditingNoteText] = useState("");
  const [isEditingNote, setIsEditingNote] = useState(false);
  const [noteSavedFeedback, setNoteSavedFeedback] = useState(false);

  // Notification Banner state
  const [notification, setNotification] = useState<{ type: 'error' | 'success' | 'info'; title: string;
message: string; suggestion?: string } | null>(null);

  // PDF Preview and Download states
  const [previewPdfUrl, setPreviewPdfUrl] = useState(null);
  const [previewPdfTitle, setPreviewPdfTitle] = useState("");
  const [previewLoadingId, setPreviewLoadingId] = useState(null);
  const [downloadLoadingId, setDownloadLoadingId] = useState(null);

  // Enlarged Image Modal state
  const [enlargedImageUrl, setEnlargedImageUrl] = useState(null);
  const [enlargedImageTitle, setEnlargedImageTitle] = useState("");

  const handlePreviewPdf = async (item: KidneyAnalysis, e?: React.MouseEvent) => {
    if (e) e.stopPropagation();
    setPreviewLoadingId(item.id);
    try {

```

```

    const url = await getReportPdfBlobUrl(item.id);
    setPreviewPdfTitle(`Diagnostic Report - ${item.patientName} || item.patientId} (${item.id})`);
    setPreviewPdfUrl(url);
  } catch (err: any) {
    const parsed = parseClinicalError(err);
    setNotification({
      type: 'error',
      title: parsed.title,
      message: parsed.message,
      suggestion: parsed.suggestion
    });
  } finally {
    setPreviewLoadingId(null);
  }
};

```

```

const handleDownloadPdf = async (item: KidneyAnalysis, e?: React.MouseEvent) => {
  if (e) e.stopPropagation();
  setDownloadLoadingId(item.id);
  try {
    await downloadReport(item.id, `kidneyvision_report_${item.id}.pdf`);
    setNotification({
      type: 'success',
      title: 'Report Downloaded',
      message: `Diagnostic PDF report for study #${item.id} downloaded successfully.`
    });
  } catch (err: any) {
    const parsed = parseClinicalError(err);
    setNotification({
      type: 'error',
      title: parsed.title,
      message: parsed.message,
      suggestion: parsed.suggestion
    });
  } finally {
    setDownloadLoadingId(null);
  }
};

```

```

const openDetails = (item: KidneyAnalysis) => {
  setSelectedAnalysis(item);
  setDossierViewMode(item.visualizationUrl ? "panel" : item.heatmapUrl ? "gradcam" : "raw");
  setEditingNoteText(item.clinicianNotes || "");
  setIsEditingNote(false);
  setNoteSavedFeedback(false);
};

```

```

async function loadAnalyses(page: number) {
  setLoading(true);
  try {
    const response = await getAnalyses(page, 10);
    setAnalyses(response.data);
    setMeta(response.meta);
    setCurrentPage(response.meta.current_page);
  } catch (e: any) {
    const parsed = parseClinicalError(e);
    setNotification({
      type: 'error',
      title: parsed.title,
      message: parsed.message,
      suggestion: parsed.suggestion
    });
  }
}

```

```

    });
  } finally {
    setLoading(false);
  }
}

useEffect(() => {
  loadAnalyses(currentPage);
}, [currentPage]);

// Handle Deletion (DELETE /analyses/:id)
const handleDelete = async (id: string, e: React.MouseEvent) => {
  e.stopPropagation(); // Avoid triggering details modal drawer
  if (confirm(`Are you sure you want to delete patient record ${id}?`)) {
    try {
      await deleteAnalysis(id);
      setAnalyses(prev => prev.filter(x => x.id !== id));
      setMeta(prev => ({ ...prev, total: Math.max(0, prev.total - 1) }));
      if (selectedAnalysis?.id === id) {
        setSelectedAnalysis(null);
      }
      setNotification({
        type: 'success',
        title: 'Record Deleted',
        message: `Patient analysis record #${id} was deleted successfully.`
      });
    } catch (err: any) {
      const parsed = parseClinicalError(err);
      setNotification({
        type: 'error',
        title: parsed.title,
        message: parsed.message,
        suggestion: parsed.suggestion
      });
    }
  }
};

// Perform search & state filtering
const filteredAnalyses = analyses.filter((item) => {
  const query = searchId.trim().toLowerCase();
  const matchesSearch = !query ||
    (item.patientId && item.patientId.toLowerCase().includes(query)) ||
    (item.patientName && item.patientName.toLowerCase().includes(query)) ||
    (item.id && String(item.id).toLowerCase().includes(query)) ||
    (item.location && item.location.toLowerCase().includes(query));

  // Status filter - supporting all 4 pathologies and triage states
  let matchesStatus = true;
  const itemDiag = item.diagnosis;
  const itemFinding = (item.pathologyFinding || item.prediction || '').toLowerCase();

  if (statusFilter === "Normal Findings") {
    matchesStatus = itemDiag === DiagnosisResult.NORMAL_FINDINGS || itemFinding.includes('normal');
  } else if (statusFilter === "Kidney Stone") {
    matchesStatus = itemDiag === DiagnosisResult.STONE || itemFinding.includes('stone') ||
    itemFinding.includes('nephro');
  } else if (statusFilter === "Renal Cyst") {
    matchesStatus = itemDiag === DiagnosisResult.CYST || itemFinding.includes('cyst');
  } else if (statusFilter === "Renal Tumor") {
    matchesStatus = itemDiag === DiagnosisResult.TUMOR || itemFinding.includes('tumor');
  }
});

```

```

    } else if (statusFilter === "Review Required") {
        matchesStatus = itemDiag === DiagnosisResult.REVIEW_REQUIRED;
    } else if (statusFilter === "Scan Rejected") {
        matchesStatus = itemDiag === DiagnosisResult.REJECTED || itemFinding.includes('reject') ||
itemFinding.includes('invalid');
    }

// Date filter
let matchesDate = true;
if (dateFilter !== "All Time") {
    const itemTime = new Date(item.createdAt).getTime();
    const now = Date.now();
    if (!isNaN(itemTime)) {
        if (dateFilter === "Last 7 Days") matchesDate = itemTime >= now - 7 * 86400000;
        else if (dateFilter === "Last 30 Days") matchesDate = itemTime >= now - 30 * 86400000;
        else if (dateFilter === "Last 90 Days") matchesDate = itemTime >= now - 90 * 86400000;
    }
}

return matchesSearch && matchesStatus && matchesDate;
});

// Relative date formatting
const formatDate = (isoString: string) => {
    try {
        const date = new Date(isoString);
        if (isNaN(date.getTime())) {
            return { date: "Unknown Date", time: "--:--" };
        }
        const months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"];
        const formattedDate = `${months[date.getMonth()]} ${date.getDate()}, ${date.getFullYear()}`;

        let hours = date.getHours();
        const minutes = date.getMinutes().toString().padStart(2, '0');
        const ampm = hours >= 12 ? 'PM' : 'AM';
        hours = hours % 12;
        hours = hours ? hours : 12;

        return {
            date: formattedDate,
            time: `${hours}:${minutes} ${ampm}`
        };
    } catch {
        return { date: "Unknown Date", time: "--:--" };
    }
};

return (
    { /* Upper header section */ }

```

Analysis History

Review past ultrasound analyses, diagnostic confidence scores, and historical patient imaging records.

```
{/* Top bar controls: Search & Filters (identical to Screen 3 Header controls) */}
```

```
{/* Search Patient ID */}
```

```
{searchId} setSearchId(e.target.value)}
/>
```

```
{/* Date range trigger dropdown */}
```

```
{ setShowDateDropdown(!showDateDropdown); setShowFilterDropdown(false); }
className="flex items-center gap-1.5 px-3 py-2 bg-white border border-[#c3c6d7] rounded-lg text-
xs font-sans text-[#131b2e] hover:bg-[#faf8ff] transition-colors cursor-pointer"
>
{dateFilter}
▼
```

```
{showDateDropdown && (
```

```
{(["All Time", "Last 7 Days", "Last 30 Days", "Last 90 Days"] as const).map((opt) => (
```

```
{
  setDateFilter(opt);
  setShowDateDropdown(false);
})
className={`w-full text-left px-3 py-2 hover:bg-[#f2f3ff] text-[#131b2e] cursor-pointer flex
  items-center justify-between ${
    dateFilter === opt ? "font-bold text-[#2563eb] bg-[#f2f3ff]" : ""
  }`}
>
{opt}
{dateFilter === opt && }
```

```

    )})

  })

  { /* Diagnosis code filter dropdown */}

  { setShowFilterDropdown(!showFilterDropdown); setShowDateDropdown(false); }}
  className="flex items-center gap-1.5 px-3 py-2 bg-white border border-[#c3c6d7] rounded-lg text-
    xs font-sans text-[#131b2e] hover:bg-[#faf8ff] transition-colors cursor-pointer"
    >

    {statusFilter}
    ▼

  {showFilterDropdown && (

    {["All Results", "Normal Findings", "Kidney Stone", "Renal Cyst", "Renal Tumor", "Review
    Required", "Scan Rejected"].map((opt) => (

      {
        setStatusFilter(opt);
        setShowFilterDropdown(false);
      }}
    )
  )
}

```

Code Source Extrait 4.6 — Historique des consultations et export PDF (src/components/AppHistory.tsx).

8. Conclusion du Frontend React

Le client web de KidneyVision AI démontre comment une interface moderne et fluide peut transformer des modèles complexes de Deep Learning en véritables assistants cliniques du quotidien. L'interaction transparente permise par le visualiseur Grad-CAM place le médecin au centre de la décision médicale.